

UNIVERSIDAD CATÓLICA DE COLOMBIA

Facultad de Ingeniería de Sistemas y Computación

DOCUMENTO TÉCNICO

Proyecto: MusicItself

Biblioteca Musical Personal con API REST

Autor: Juan Diego Lozano

Repositorio: <https://github.com/2naain/preparcf>

Despliegue: <https://musiicitsself.onrender.com>

Bogotá D.C., Colombia — 2026

1. Introducción

El presente documento constituye el informe técnico del proyecto MusicItself, una aplicación web orientada a la gestión de una biblioteca musical personal. El sistema permite al usuario administrar canciones, artistas y listas de reproducción (playlists) mediante una interfaz web y una API REST documentada con Swagger UI.

El desarrollo del proyecto integra conocimientos de programación orientada al backend con el framework FastAPI (Python), gestión de bases de datos relacionales mediante SQLAlchemy y PostgreSQL (Neon DB), almacenamiento de imágenes en la nube con Supabase Storage, y la generación de interfaces web dinámicas mediante plantillas Jinja2.

El sistema se encuentra actualmente desplegado en la plataforma Render, accesible públicamente en la URL: musiicitsself.onrender.com, lo que demuestra la viabilidad de un despliegue continuo basado en un repositorio de control de versiones alojado en GitHub.

2. Objetivos

2.1 Objetivo General

Desarrollar una aplicación web full-stack que permita la gestión integral de una biblioteca musical personal, mediante una API REST robusta complementada con un frontend dinámico, persistencia de datos en la nube y capacidad de despliegue continuo.

2.2 Objetivos Específicos

- Diseñar e implementar un modelo de datos relacional que soporte las entidades Canción, Artista y Playlist con sus respectivas relaciones.
- Exponer los recursos del sistema a través de una API REST con operaciones CRUD completas para cada entidad.
- Integrar el almacenamiento remoto de imágenes mediante el servicio Supabase Storage.
- Implementar un frontend funcional con plantillas Jinja2 que consuma directamente los servicios del backend.
- Documentar automáticamente la API mediante la interfaz Swagger UI proporcionada por FastAPI.
- Desplegar la aplicación en la nube utilizando Render como plataforma de alojamiento.

3. Arquitectura del Sistema

La arquitectura del proyecto MusicItself corresponde al patrón monolítico integrado, en el cual el backend (lógica de negocio y API REST) y el frontend (plantillas HTML renderizadas en el servidor) coexisten en la misma aplicación. Este diseño es adecuado para proyectos de complejidad media, reduciendo la sobrecarga de comunicación entre servicios independientes.

El flujo general del sistema es el siguiente: el cliente (navegador web) realiza solicitudes HTTP al servidor FastAPI; el servidor procesa la solicitud invocando las operaciones correspondientes sobre la base de datos PostgreSQL alojada en Neon DB; en caso de operaciones con imágenes, el archivo es almacenado en Supabase Storage y se retorna una URL pública; finalmente, el servidor retorna una respuesta en formato JSON (para endpoints de la API) o una página HTML renderizada (para los endpoints de interfaz gráfica).

3.1 Tecnologías Empleadas

Tecnología	Rol en el sistema	Versión
Python 3.12	Lenguaje principal del backend	3.12
FastAPI	Framework web y API REST	0.115.12
SQLModel	ORM y validación de modelos	0.0.22
PostgreSQL (Neon DB)	Base de datos relacional en la nube	—
Supabase Storage	Almacenamiento de imágenes en la nube	2.15.1
Jinja2	Motor de plantillas HTML	3.1.6
Uvicorn	Servidor ASGI	0.34.0
Render	Plataforma de despliegue en la nube	—
GitHub	Control de versiones	—

4. Modelo de Datos

El sistema gestiona tres entidades principales relacionadas entre sí. A continuación se describe cada una con sus atributos y restricciones.

4.1 Entidad: Canción (SongID)

Campo	Tipo	Restricción	Descripción
id	Integer	PK, Auto	Identificador único de la canción
title	String	2–100 chars	Título de la canción
genre	Enum (SongGenre)	Opcional	Género musical
duration	Integer	1–10000 seg	Duración en segundos

Campo	Tipo	Restricción	Descripción
artist_id	Integer	FK → ArtistID.id	Artista asociado
in_playlist	Boolean	Default: False	Indica si está en playlist
is_active	Boolean	Default: True	Borrado lógico
image_url	String	Máx. 500 chars	URL de imagen en Supabase

Los géneros musicales válidos están definidos mediante un enumerado (SongGenre): pop, rock, jazz, hiphop, electronica, clasica, reggaeton y metal.

4.2 Entidad: Artista (ArtistID)

Campo	Tipo	Restricción	Descripción
id	Integer	PK, Auto	Identificador único del artista
name	String	2–100 chars	Nombre del artista
image_url	String	Máx. 500 chars	URL de imagen en Supabase
is_active	Boolean	Default: True	Borrado lógico

4.3 Entidad: Playlist (PlaylistID)

Campo	Tipo	Restricción	Descripción
id	Integer	PK, Auto	Identificador único de la playlist
name	String	2–100 chars	Nombre de la playlist
description	String	Máx. 500 chars	Descripción opcional
is_active	Boolean	Default: True	Borrado lógico

4.4 Tabla de relación: PlaylistSong

La relación muchos-a-muchos entre Playlist y Canción se implementa mediante la tabla intermedia PlaylistSong, cuya clave primaria compuesta está formada por playlist_id (FK → PlaylistID.id) y song_id (FK → SongID.id). Esta estructura garantiza la integridad referencial y evita duplicados de canciones dentro de una misma playlist.

5. Descripción de la API REST

La API REST del sistema expone tres módulos funcionales: Songs API, Artists API y Playlists API, todos disponibles bajo el prefijo base de la aplicación desplegada. La documentación interactiva es accesible desde el endpoint /docs (Swagger UI) generado automáticamente por FastAPI.

5.1 Songs API

Método	Ruta	Descripción
GET	/song	Retorna listado de todas las canciones activas
POST	/song	Crea una nueva canción con datos y carga de imagen
GET	/song/{id}	Retorna una canción específica por ID
PATCH	/song/{id}	Actualiza parcialmente una canción existente
DELETE	/song/{id}	Realiza borrado lógico de una canción

5.2 Artists API

Método	Ruta	Descripción
GET	/artist	Retorna todos los artistas activos
POST	/artist	Registra un nuevo artista en el sistema
GET	/artist/{id}	Obtiene los datos de un artista por ID
PATCH	/artist/{id}	Actualiza los datos de un artista
DELETE	/artist/{id}	Borrado lógico de un artista

5.3 Playlists API

Método	Ruta	Descripción
GET	/playlist	Lista todas las playlists activas
POST	/playlist	Crea una nueva playlist
GET	/playlist/{id}	Obtiene los datos de una playlist
PATCH	/playlist/{id}	Actualiza una playlist existente

Método	Ruta	Descripción
DELETE	/playlist/{id}	Borrado lógico de una playlist
POST	/playlist/{playlist_id}/song/{song_id}	Agrega una canción a una playlist
DELETE	/playlist/{playlist_id}/song/{song_id}	Elimina una canción de una playlist
GET	/playlist/{id}/songs	Lista todas las canciones de una playlist

6. Módulo de Almacenamiento de Imágenes

El sistema implementa dos estrategias de almacenamiento de imágenes: almacenamiento local (save_img_local) para entornos de desarrollo, y almacenamiento remoto en Supabase Storage (save_img_remote) para el entorno de producción.

El flujo de almacenamiento remoto es el siguiente: el archivo de imagen es recibido en el endpoint correspondiente como un objeto UploadFile; se valida que el tipo MIME corresponda a una imagen (image/*); el contenido binario es enviado al bucket configurado en Supabase mediante el cliente oficial de Python; finalmente, se obtiene la URL pública del archivo almacenado, la cual es persistida en la columna image_url de la entidad correspondiente.

7. Interfaz de Usuario (Frontend)

El frontend de la aplicación se implementa mediante el motor de plantillas Jinja2, integrado nativamente con FastAPI. Las vistas son renderizadas en el servidor (SSR - Server Side Rendering), lo que simplifica la arquitectura al evitar la necesidad de un framework JavaScript independiente.

El diseño visual adopta una paleta oscura con acentos en tonos morados y cian, logrando una estética coherente para una aplicación de gestión musical. La estructura de la interfaz se organiza mediante una barra de navegación superior con acceso a las secciones Inicio, Canciones, Artistas y Playlists, además de un buscador global.

7.1 Vista de Inicio (Home)

La página de inicio presenta un panel estadístico con el conteo total de canciones, artistas y playlists registradas en el sistema. Adicionalmente, muestra las seis canciones registradas más recientemente en formato de tarjetas, con su título, duración, identificador y género. La Figura 1 ilustra esta vista.

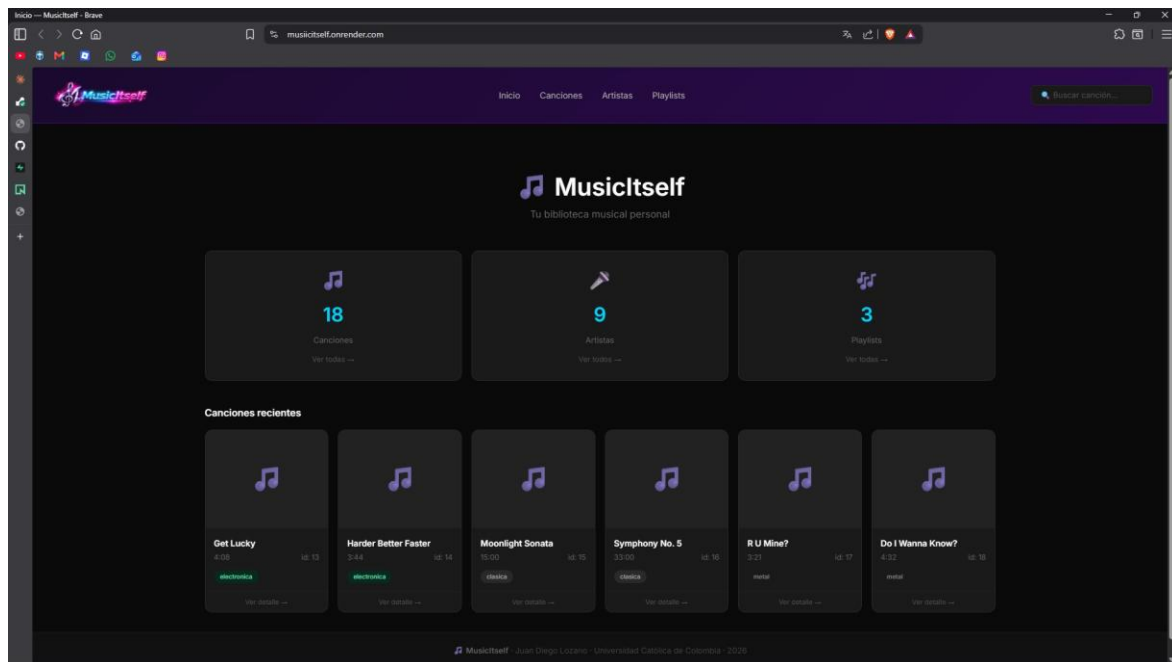


Figura 1. Vista de inicio de MusicItself — Panel estadístico y canciones recientes.

7.2 Vista de Canciones

La vista de canciones presenta la totalidad del catálogo musical registrado en el sistema, organizado en una cuadrícula de tarjetas. Cada tarjeta exhibe el título, duración formateada (mm:ss), identificador, género y acceso al detalle de la canción. La interfaz incluye el botón '+ Agregar canción', que redirige al formulario de creación. La Figura 2 muestra esta vista.

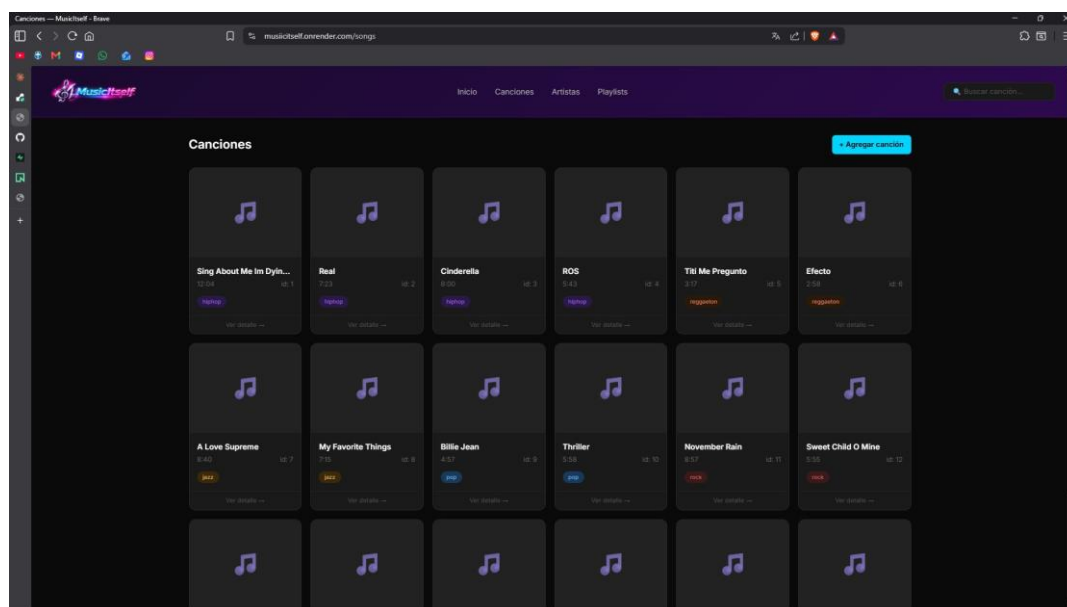


Figura 2. Vista del catálogo de canciones — Listado completo con tarjetas informativas.

8. Documentación de la API — Swagger UI

FastAPI genera de forma automática una interfaz de documentación interactiva basada en el estándar OpenAPI 3.0. Esta interfaz, accesible en el endpoint `/docs`, permite explorar y probar todos los endpoints disponibles sin necesidad de herramientas externas como Postman o curl.

La Figura 3 muestra la interfaz Swagger UI con los tres módulos de la API desplegados: Songs API, Artists API y Playlists API, evidenciando la completitud del sistema de operaciones CRUD implementado.

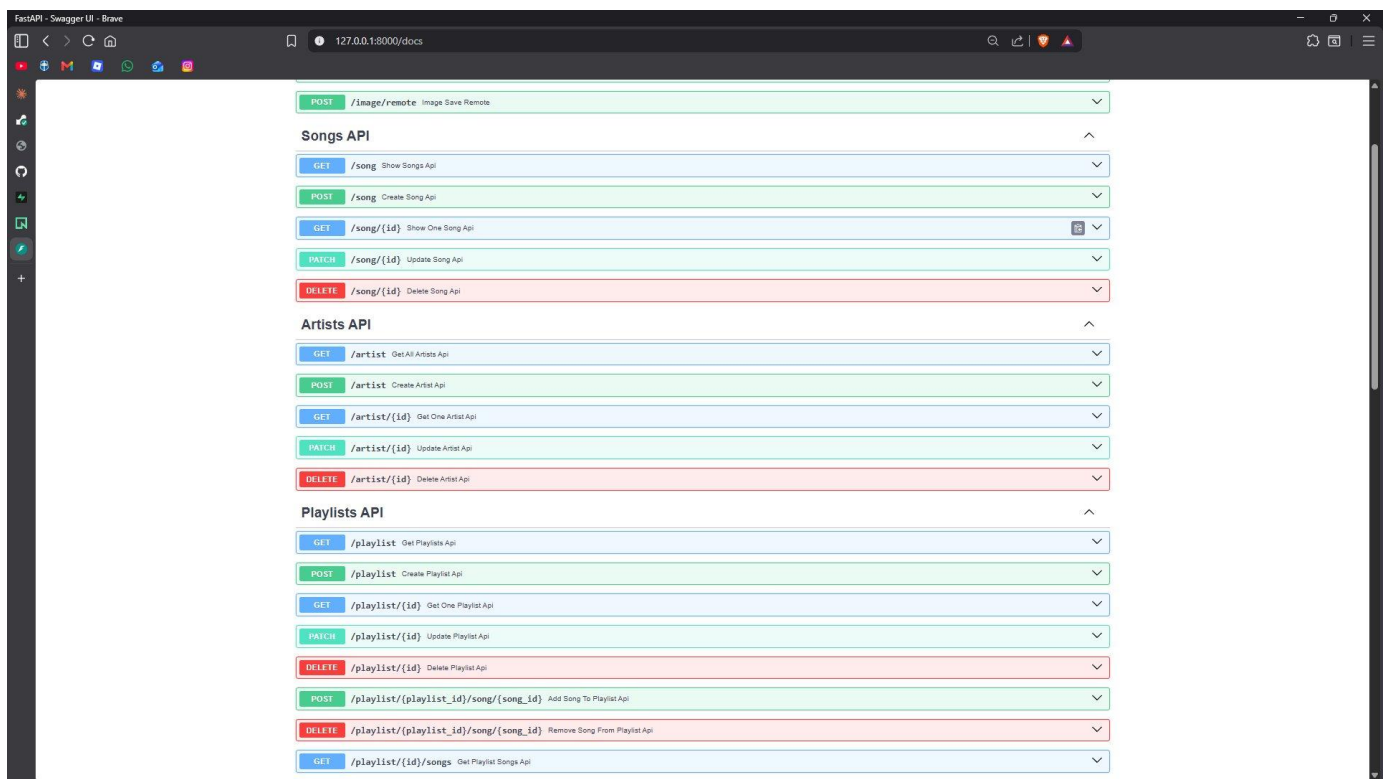


Figura 3. Interfaz Swagger UI — Documentación interactiva de los endpoints de la API REST.

Cabe destacar que, además de los tres módulos principales, el sistema expone endpoints auxiliares para la carga de imágenes (`/image/local` y `/image/remote`), los cuales facilitan la gestión del contenido multimedia asociado a canciones y artistas.

9. Estructura del Proyecto

La organización del repositorio sigue una arquitectura por capas, separando las responsabilidades de cada módulo de la siguiente manera:

Ruta / Archivo	Responsabilidad
main.py	Punto de entrada de la aplicación; definición de todos los endpoints de la API REST y del frontend.
db.py	Configuración de la conexión a la base de datos PostgreSQL (Neon DB) y gestión del ciclo de vida de las sesiones.
utils.py	Funciones utilitarias para el almacenamiento de imágenes (local y remoto mediante Supabase).
models/song.py	Definición de los modelos SQLAlchemy para la entidad Canción: SongBase, SongID, SongUpdate.
models/artist.py	Definición de los modelos para la entidad Artista: ArtistBase, ArtistID.
models/playlist.py	Definición de los modelos para la entidad Playlist: PlaylistBase, PlaylistID.
models/song_genres.py	Enumerado SongGenre con los géneros musicales válidos del sistema.
models/playlist_song.py	Tabla intermedia PlaylistSong para la relación muchos-a-muchos entre Playlist y Canción.
operations/operations_song_db.py	Capa de acceso a datos para canciones: CRUD completo con soporte de borrado lógico y búsqueda por título.
operations/operations_artist_db.py	Capa de acceso a datos para artistas: CRUD completo con búsqueda por nombre.
operations/operations_playlist_db.py	Capa de acceso a datos para playlists: CRUD y gestión de canciones asociadas.
templates/	Directorio con plantillas Jinja2 para las vistas HTML del frontend.
requirements.txt	Lista de dependencias Python del proyecto para reproducibilidad del entorno.
runtime.txt	Especificación de la versión de Python para el entorno de despliegue en Render.

10. Despliegue e Infraestructura

El despliegue de la aplicación se realiza sobre la plataforma Render, que permite alojar aplicaciones web conectadas directamente a un repositorio de GitHub, habilitando un flujo de integración continua básico (CI/CD). Al detectar cambios en la rama principal del repositorio, Render construye y despliega automáticamente la nueva versión de la aplicación.

La infraestructura del sistema en producción se compone de los siguientes servicios externos:

- **Render (Web Service):** Aloja la aplicación FastAPI y expone los endpoints públicamente bajo el dominio `musiicitsself.onrender.com`.
- **Neon DB (PostgreSQL):** Servicio de base de datos relacional serverless que gestiona la persistencia de las entidades del sistema (canciones, artistas, playlists y relaciones).
- **Supabase Storage:** Servicio de almacenamiento de objetos en la nube que aloja las imágenes de canciones y artistas, retornando URLs públicas de acceso.

La variable de entorno `DATABASE_URL_NEON` configura la cadena de conexión a la base de datos, mientras que `SUPABASE_URL` y `SUPABASE_KEY` autentican el cliente de Supabase. La variable `ENV` controla si se ejecuta la creación automática de tablas al iniciar la aplicación (modo dev) o si se omite dicho paso (modo producción).

11. Consideraciones Técnicas y Decisiones de Diseño

11.1 Borrado Lógico

Las entidades Canción, Artista y Playlist implementan borrado lógico mediante el atributo `is_active` (Boolean). Esto significa que los registros no son eliminados físicamente de la base de datos; en cambio, son marcados como inactivos. Esta decisión preserva la integridad histórica de los datos y facilita posibles procesos de auditoría o recuperación.

11.2 Formato de Duración

La duración de las canciones se almacena en segundos (tipo Integer) en la base de datos. La conversión al formato legible `mm:ss` se realiza mediante el filtro personalizado `duration` registrado en el entorno de plantillas Jinja2, separando las responsabilidades de presentación del modelo de datos.

11.3 Validación con SQLAlchemy

El uso de SQLAlchemy integra en un único modelo tanto la definición del esquema de la tabla de base de datos (mediante SQLAlchemy) como la validación de datos de entrada (mediante Pydantic). Esto elimina la redundancia de definir modelos separados para la base de datos y para la API, reduciendo la superficie de error.

11.4 Búsqueda con ILIKE

Las funciones de listado de canciones, artistas y playlists soportan un parámetro de búsqueda opcional (q). La búsqueda se implementa mediante el operador ILIKE de PostgreSQL, que realiza comparaciones de cadenas insensibles a mayúsculas/minúsculas con coincidencia parcial (patrón %q%), ofreciendo una experiencia de búsqueda más flexible para el usuario.

12. Conclusiones

El proyecto MusicItself demuestra la factibilidad de construir una aplicación web full-stack funcional y desplegada en producción utilizando exclusivamente tecnologías de código abierto del ecosistema Python. La combinación de FastAPI, SQLAlchemy y Jinja2 permite desarrollar de forma ágil tanto la API REST como la interfaz de usuario, manteniendo una base de código coherente y con reducida deuda técnica.

La integración con servicios externos (Neon DB, Supabase, Render) evidencia la viabilidad de las arquitecturas cloud-native para proyectos académicos, reduciendo significativamente la complejidad operacional de la infraestructura en comparación con despliegues tradicionales en servidores propios.

La documentación automática generada por FastAPI mediante Swagger UI constituye un valor agregado relevante, facilitando la comprensión, prueba y consumo de la API por parte de terceros sin necesidad de documentación adicional.

Como trabajo futuro, se identifican oportunidades de mejora en las siguientes áreas: implementación de autenticación de usuarios mediante JWT; paginación de los resultados de los listados; migración del frontend a un framework JavaScript moderno para mejorar la reactividad de la interfaz; e implementación de pruebas automatizadas (unitarias e integración) para garantizar la robustez del sistema ante cambios.

Referencias

- FastAPI. (2024). FastAPI Documentation. <https://fastapi.tiangolo.com/>
- SQLModel. (2024). SQLAlchemy Documentation. <https://sqlmodel.tiangolo.com/>
- Supabase. (2024). Supabase Python Client Documentation. <https://supabase.com/docs/reference/python/introduction>
- Neon. (2024). Neon Serverless Postgres Documentation. <https://neon.tech/docs>
- Render. (2024). Render Deploy Documentation. <https://render.com/docs>
- Pydantic. (2024). Pydantic V2 Documentation. <https://docs.pydantic.dev/latest/>
- Swagger. (2024). OpenAPI Specification 3.0. <https://swagger.io/specification/>